

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 14
“PENGURUTAN PADA DATA TERSTRUKTUR”



DISUSUN OLEH:
Dharma Chandra Viriya
109082500052
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

CONTOH SOAL

1. Latihan1

Source Code:

Output:

Deskripsi Program:

SOAL LATIHAN

SELECTION SORT

Statement selection sort

1. Latihan1

Source Code:

```
package main

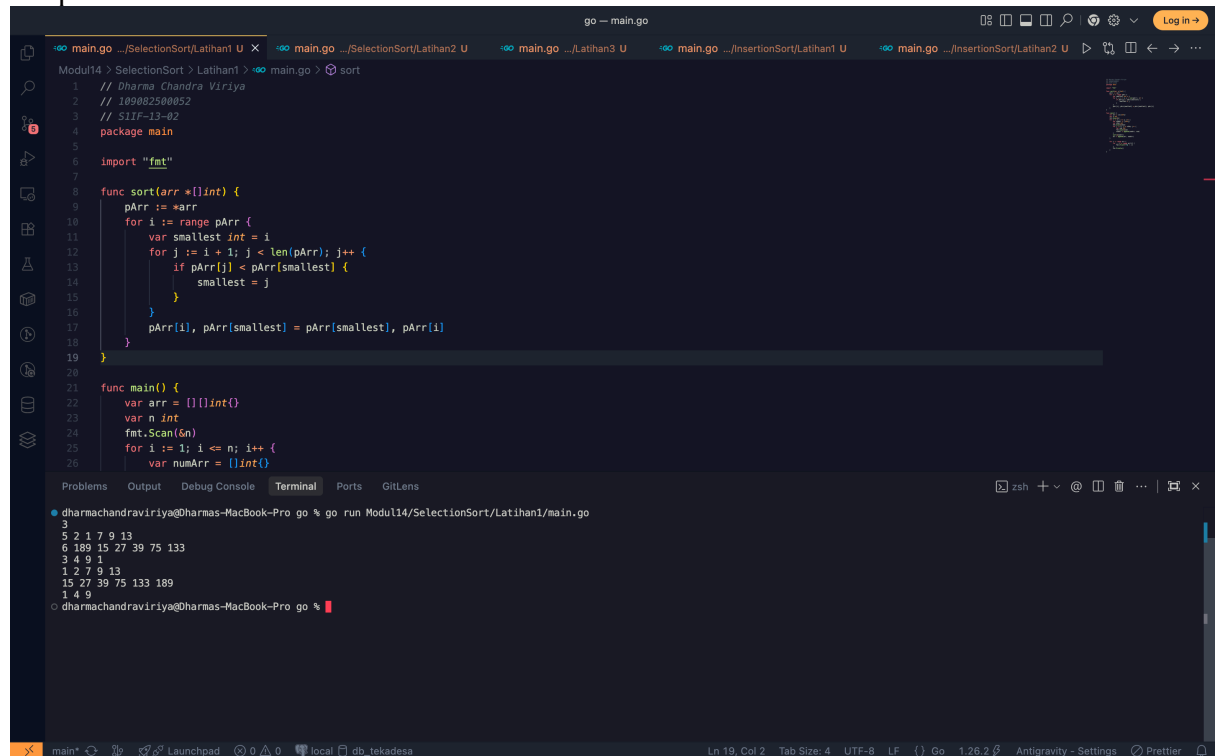
import "fmt"

func sort(arr *[]int) {
    pArr := *arr
    for i := range pArr {
        var smallest int = i
        for j := i + 1; j < len(pArr); j++ {
            if pArr[j] < pArr[smallest] {
                smallest = j
            }
        }
        pArr[i], pArr[smallest] = pArr[smallest], pArr[i]
    }
}

func main() {
    var arr = [][]int{}
    var n int
    fmt.Scan(&n)
    for i := 1; i <= n; i++ {
        var numArr = []int{}
        var nSub int
        fmt.Scan(&nSub)
        for j := 1; j <= nSub; j++ {
            var num int
            fmt.Scan(&num)
            numArr = append(numArr, num)
        }
        sort(&numArr)
        arr = append(arr, numArr)
    }

    for i := range arr {
        for _, v := range arr[i] {
            fmt.Printf("%d ", v)
        }
        fmt.Println()
    }
}
```

Output:



The screenshot shows a Go IDE with a file named `main.go` containing a Selection Sort implementation. The code is as follows:

```
1 // Dharma Chandra Viriya
2 // 109002500052
3 // S11F-13-02
4 package main
5
6 import "fmt"
7
8 func sort(arr *[]int) {
9     pArr := *arr
10    for i := range pArr {
11        var smallest int = i
12        for j := i + 1; j < len(pArr); j++ {
13            if pArr[j] < pArr[smallest] {
14                smallest = j
15            }
16        }
17        pArr[i], pArr[smallest] = pArr[smallest], pArr[i]
18    }
19 }
20
21 func main() {
22     var arr = []int{}
23     var n int
24     fmt.Scan(&n)
25     for i := 1; i <= n; i++ {
26         var numArr = []int{}
27     }
```

The terminal output shows the execution of the program with the input `5` and the resulting sorted array:

```
dharmachandraviriya@Dharmas-MacBook-Pro % go run Modul14/SelectionSort/Latihan1/main.go
5
2 1 7 9 13
6 189 15 27 39 75 133
3 4 9 1
1 2 7 9 13
15 27 39 75 133 189
1 4 9
```

Deskripsi Program:

Program ini digunakan untuk mengurutkan data nomor rumah pada beberapa daerah berdasarkan urutan menaik (ascending). Masukan diawali dengan jumlah daerah yang akan diproses, kemudian untuk setiap daerah dimasukkan banyaknya nomor rumah beserta nilai nomor rumah tersebut. Setelah seluruh data dibaca, program mengurutkan nomor rumah pada masing-masing daerah dan menampilkan hasil pengurutan sesuai urutan daerah pada masukan. Secara teknis, program menggunakan struktur data slice dua dimensi (`[][]int`) untuk menyimpan kumpulan nomor rumah dari setiap daerah. Proses pengurutan dilakukan melalui fungsi `sort()` yang menerapkan algoritma Selection Sort. Pada setiap iterasi, algoritma mencari elemen dengan nilai terkecil pada bagian array yang belum terurut, kemudian menukarnya dengan elemen pada posisi saat ini. Proses ini dilakukan berulang hingga seluruh elemen tersusun secara menaik. Implementasi Selection Sort pada program menggunakan dua buah perulangan bersarang. Perulangan pertama menentukan posisi yang akan diisi dengan elemen terkecil, sedangkan perulangan kedua digunakan untuk mencari indeks elemen terkecil pada sisa data yang belum terurut. Setelah indeks elemen terkecil ditemukan, dilakukan proses pertukaran (swap) menggunakan assignment berganda. Algoritma ini memiliki kompleksitas waktu sebesar $O(n^2)$ dan kompleksitas ruang $O(1)$ karena proses pengurutan dilakukan langsung pada data yang ada tanpa memerlukan struktur penyimpanan tambahan yang signifikan.

2. Latihan2

Source Code:

```
package main

import "fmt"

func oddEven(arr []int) ([]int, []int) {
    var odd []int
    var even []int
    for _, v := range arr {
        if v%2 != 0 {
            odd = append(odd, v)
        } else {
            even = append(even, v)
        }
    }
    return odd, even
}

func sort(arr *[]int, isASC bool) {
    pArr := *arr
    for i := 0; i < len(pArr)-1; i++ {
        smallest := i
        for j := i + 1; j < len(pArr); j++ {
            if isASC {
                if pArr[j] < pArr[smallest] {
                    smallest = j
                }
            } else {
                if pArr[j] > pArr[smallest] {
                    smallest = j
                }
            }
        }
        pArr[i], pArr[smallest] = pArr[smallest], pArr[i]
    }
}
```

```

func main() {
    var arr = [][]int{}
    var n int
    fmt.Scan(&n)
    for i := 1; i <= n; i++ {
        var numArr = []int{}
        var nSub int
        fmt.Scan(&nSub)
        for j := 1; j <= nSub; j++ {
            var num int
            fmt.Scan(&num)
            numArr = append(numArr, num)
        }
        oddArr, evenArr := oddEven(numArr)
        sort(&oddArr, true)
        sort(&evenArr, false)
        sortedArr := append(oddArr, evenArr...)
        arr = append(arr, sortedArr)
    }
    for i := range arr {
        for _, v := range arr[i] {
            fmt.Printf("%d ", v)
        }
        fmt.Println()
    }
}

```

Output:

The screenshot shows a Go IDE with the following source code in the editor:

```

1 // Dharma Chandra Viriya
2 // 109082500052
3 // SIIF-13-02
4 package main
5
6 import "fmt"
7
8 func oddEven(arr []int) ([]int, []int) {
9     var odd []int
10    var even []int
11    for _, v := range arr {
12        if v%2 != 0 {
13            odd = append(odd, v)
14        } else {
15            even = append(even, v)
16        }
17    }
18    return odd, even
19 }
20
21 func sort(arr *[]int, isASC bool) {
22     pArr := *arr
23     for i := 0; i < len(pArr)-1; i++ {
24         smallest := i
25         for j := i + 1; j < len(pArr); j++ {
26             if isASC {

```

The terminal output shows the execution of the program:

```

dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul14/SelectionSort/Latihan2/main.go
3
5 2 1 7 9 13
6 189 15 27 39 75 133
3 4 9 1
1 7 9 13 2
15 27 39 75 133 189
1 9 4

```

Deskripsi Program:

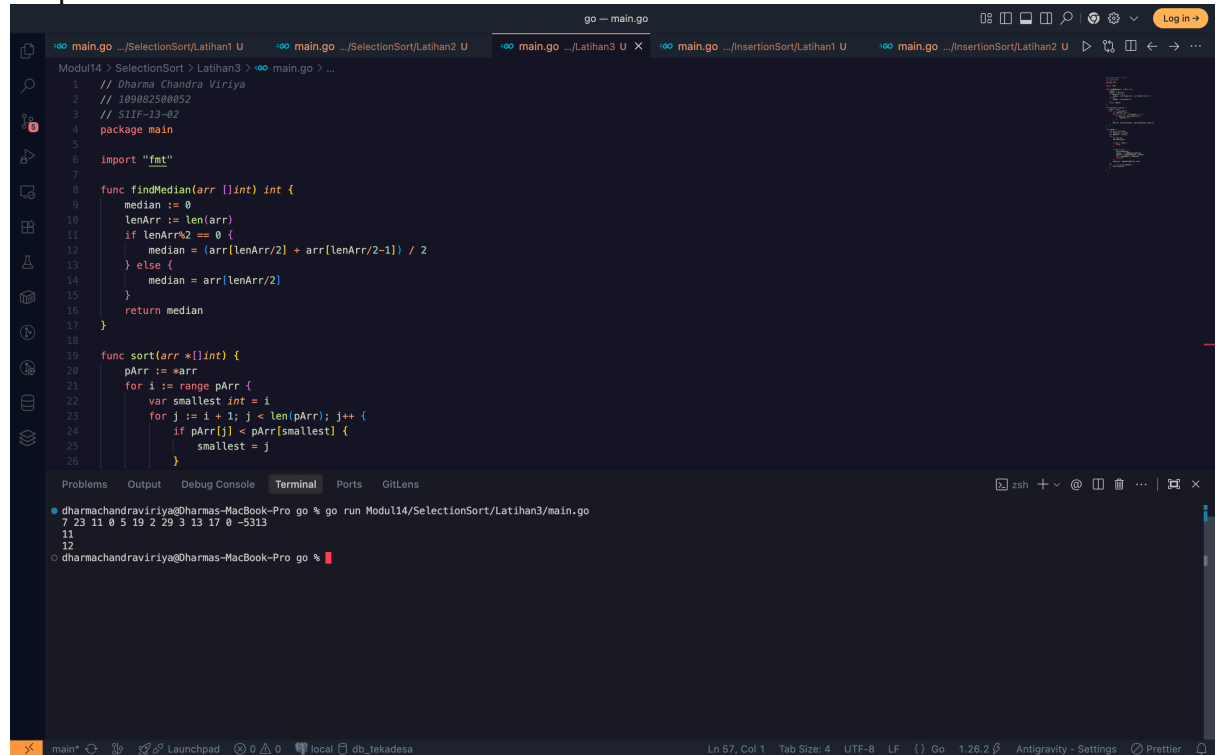
Program ini digunakan untuk mengelompokkan dan mengurutkan nomor rumah pada beberapa daerah berdasarkan paritas bilangan. Setiap nomor rumah dipisahkan menjadi dua kelompok, yaitu bilangan ganjil dan bilangan genap. Setelah proses pemisahan, nomor rumah ganjil diurutkan secara menaik (ascending), sedangkan nomor rumah genap diurutkan secara menurun (descending). Hasil akhir yang ditampilkan untuk setiap daerah adalah seluruh nomor rumah ganjil yang telah terurut, kemudian diikuti oleh seluruh nomor rumah genap yang juga telah terurut. Secara teknis, program memanfaatkan fungsi `oddEven()` untuk memisahkan data ke dalam dua slice berbeda berdasarkan hasil operasi modulus (`% 2`). Bilangan dengan sisa bagi satu dimasukkan ke dalam slice ganjil, sedangkan bilangan dengan sisa bagi nol dimasukkan ke dalam slice genap. Setelah proses pemisahan selesai, masing-masing slice diurutkan secara independen menggunakan fungsi `sort()`. Fungsi `sort()` mengimplementasikan algoritma Selection Sort. Pada setiap iterasi, algoritma mencari elemen terkecil atau terbesar pada bagian array yang belum terurut, bergantung pada nilai parameter `isASC`. Jika `isASC` bernilai `true`, data diurutkan secara menaik dengan memilih elemen terkecil pada setiap iterasi. Sebaliknya, jika `isASC` bernilai `false`, data diurutkan secara menurun dengan memilih elemen terbesar. Setelah indeks elemen yang sesuai ditemukan, dilakukan pertukaran posisi (swap) dengan elemen pada indeks saat ini. Data dari setiap daerah disimpan dalam slice dua dimensi (`[][]int`) sehingga hasil pengolahan masing-masing daerah dapat dipertahankan dan dicetak kembali sesuai urutan masukan. Algoritma Selection Sort yang digunakan memiliki kompleksitas waktu $O(n^2)$ dengan kompleksitas ruang $O(1)$, karena proses pengurutan dilakukan secara langsung pada array tanpa memerlukan struktur data tambahan yang signifikan.

3. Latihan3

Source Code:

```
package main
import "fmt"
func findMedian(arr []int) int {
    median := 0
    lenArr := len(arr)
    if lenArr%2 == 0 {
        median = (arr[lenArr/2] + arr[lenArr/2-1]) / 2
    } else {
        median = arr[lenArr/2]
    }
    return median
}
func sort(arr *[]int) {
    pArr := *arr
    for i := range pArr {
        var smallest int = i
        for j := i + 1; j < len(pArr); j++ {
            if pArr[j] < pArr[smallest] {
                smallest = j
            }
        }
        pArr[i], pArr[smallest] = pArr[smallest], pArr[i]
    }
}
func main() {
    var arr = [][]int{}
    var subArray = []int{}
    var medians = []int{}
    for {
        var num int
        fmt.Scan(&num)
        if num == -5313 {
            break
        }
        if num == 0 {
            sort(&subArray)
            median := findMedian(subArray)
            medians = append(medians, median)
            arr = append(arr, subArray)
            continue
        }
        subArray = append(subArray, num)
    }
    for _, v := range medians {
        fmt.Println(v)
    }
}
```

Output:



The screenshot shows a Go IDE with a dark theme. The main editor displays a Go program for finding the median of an array using Selection Sort. The code includes comments, package declarations, imports, and functions for finding the median and sorting the array. The terminal at the bottom shows the command to run the program and its output.

```
1 // Dharma Chandra Viriya
2 // 109002500052
3 // S11F-13-02
4 package main
5
6 import "fmt"
7
8 func findMedian(arr []int) int {
9     median := 0
10    lenArr := len(arr)
11    if lenArr%2 == 0 {
12        median = (arr[lenArr/2] + arr[lenArr/2-1]) / 2
13    } else {
14        median = arr[lenArr/2]
15    }
16    return median
17 }
18
19 func sort(arr *[]int) {
20    pArr := *arr
21    for i := range pArr {
22        var smallest int = i
23        for j := i + 1; j < len(pArr); j++ {
24            if pArr[j] < pArr[smallest] {
25                smallest = j
26            }
27        }
28        pArr[i], pArr[smallest] = pArr[smallest], pArr[i]
29    }
30    *arr = pArr
31 }
```

Terminal Output:

```
dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul14/SelectionSort/Latihan3/main.go
7 23 11 0 5 19 2 29 3 13 17 0 -5313
11
12
dharmachandraviriya@Dharmas-MacBook-Pro go %
```

Deskripsi Program:

Program ini digunakan untuk menentukan nilai median dari sekumpulan data bilangan bulat positif yang dibaca secara bertahap. Setiap kali program membaca nilai 0, seluruh data yang telah terkumpul hingga saat tersebut diurutkan dan nilai mediannya dihitung, kemudian hasil median disimpan untuk ditampilkan. Proses pembacaan data berlanjut hingga ditemukan nilai penanda akhir, yaitu -5313. Secara teknis, program menyimpan seluruh bilangan yang dibaca ke dalam sebuah slice. Ketika nilai 0 ditemukan, fungsi `sort()` dipanggil untuk mengurutkan data menggunakan algoritma Selection Sort. Algoritma ini bekerja dengan mencari elemen terkecil pada bagian array yang belum terurut, kemudian menukarnya dengan elemen pada posisi saat ini hingga seluruh data tersusun secara menaik. Setelah proses pengurutan selesai, fungsi `findMedian()` digunakan untuk menentukan nilai median berdasarkan jumlah elemen yang tersedia. Apabila jumlah data ganjil, median diperoleh dari elemen yang berada tepat di tengah array. Sebaliknya, jika jumlah data genap, median dihitung sebagai rata-rata dua elemen tengah dan hasil pembagian bilangan bulat secara otomatis dibulatkan ke bawah sesuai ketentuan soal. Nilai median yang diperoleh kemudian disimpan ke dalam slice `medians` dan dicetak setelah seluruh proses pembacaan data selesai. Implementasi pengurutan menggunakan Selection Sort memiliki kompleksitas waktu $O(n^2)$ karena membutuhkan proses pencarian elemen minimum pada setiap iterasi. Sementara itu, proses pencarian median setelah data terurut dapat dilakukan dalam waktu $O(1)$ karena posisi elemen median dapat ditentukan secara langsung berdasarkan ukuran array.

INSERTION SORT

Statement insertion sort

1. Latihan1

Source Code:

```
package main
import "fmt"
func sort(arr *[]int) {
    pArr := *arr
    for i := 1; i < len(pArr); i++ {
        for j := i; j > 0; j-- {
            if pArr[j] < pArr[j-1] {
                temp := pArr[j]
                pArr[j] = pArr[j-1]
                pArr[j-1] = temp
            }
        }
    }
}
func main() {
    var isSorted bool = true
    var arr = []int{}
    for {
        var num int
        fmt.Scan(&num)
        if num < 0 {
            break
        }
        arr = append(arr, num)
    }
    sort(&arr)
    gap := 0
    if len(arr) > 1 {
        gap = arr[1] - arr[0]
        for i := 0; i < len(arr)-1; i++ {
            if arr[i]+gap != arr[i+1] {
                isSorted = false
                break
            }
        }
        for _, v := range arr {
            fmt.Printf("%d ", v)
        }
        fmt.Println()
    } else {
        isSorted = false
    }
}
```

```

    if isSorted {
        fmt.Printf("Data berjarak %d\n", gap)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

```

Output:

```

// Dharma Chandra Viriya
// 109082500052
// SIIF-13-02
package main

import "fmt"

func sort(arr *[]int) {
    pArr := *arr
    for i := 1; i < len(pArr); i++ {
        for j := i; j > 0; j-- {
            if pArr[j] < pArr[j-1] {
                temp := pArr[j]
                pArr[j] = pArr[j-1]
                pArr[j-1] = temp
            }
        }
    }
}

func main() {
    var isSorted bool = true
    var arr = []int{}
    for {
        var num int
        fmt.Scan(&num)
    }
}

```

```

dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul14/InsertionSort/Latihan1/main.go
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul14/InsertionSort/Latihan1/main.go
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
dharmachandraviriya@Dharmas-MacBook-Pro go %

```

Deskripsi Program:

Program ini digunakan untuk membaca sekumpulan bilangan bulat non-negatif, mengurutkannya menggunakan algoritma Insertion Sort, kemudian memeriksa apakah setiap elemen pada data yang telah terurut memiliki selisih yang sama dengan elemen sebelumnya. Input diakhiri dengan bilangan negatif yang tidak disimpan ke dalam array. Keluaran program berupa data yang telah terurut dan informasi apakah data memiliki jarak yang tetap atau tidak. Secara teknis, data disimpan dalam sebuah slice integer dan diurutkan menggunakan fungsi `sort()` yang menerapkan algoritma Insertion Sort. Algoritma ini bekerja dengan menyisipkan setiap elemen ke posisi yang tepat pada bagian array yang sudah terurut melalui proses perbandingan dan pertukaran elemen ke arah kiri hingga urutannya benar. Setelah pengurutan selesai, program menghitung selisih dua elemen pertama sebagai nilai acuan (`gap`). Selanjutnya, setiap pasangan elemen yang berurutan diperiksa. Jika seluruh selisih bernilai sama dengan `gap`, maka program menampilkan pesan "Data berjarak x". Jika terdapat selisih yang berbeda, program menampilkan pesan "Data berjarak tidak tetap". Algoritma Insertion Sort yang digunakan memiliki kompleksitas waktu $O(n^2)$ dan kompleksitas ruang $O(1)$.

2. Latihan2

Source Code:

```
package main

import "fmt"

const nMax = 7919

type Buku struct {
    ID      string
    Judul   string
    Penulis string
    Penerbit string
    Eksemplar int
    Tahun   int
    Rating  int
}

type DaftarBuku [nMax]Buku

/*
I.S. sejumlah n data buku telah siap pada masukan
F.S. pustaka berisi n data buku
*/
func DaftarkanBuku(pustaka *DaftarBuku, n int) {
    for i := 0; i < n; i++ {
        fmt.Printf("\nData Buku %d\n", i+1)

        fmt.Print("ID      : ")
        fmt.Scan(&(*pustaka)[i].ID)

        fmt.Print("Judul   : ")
        fmt.Scan(&(*pustaka)[i].Judul)

        fmt.Print("Penulis : ")
        fmt.Scan(&(*pustaka)[i].Penulis)

        fmt.Print("Penerbit : ")
        fmt.Scan(&(*pustaka)[i].Penerbit)

        fmt.Print("Eksemplar : ")
        fmt.Scan(&(*pustaka)[i].Eksemplar)

        fmt.Print("Tahun   : ")
        fmt.Scan(&(*pustaka)[i].Tahun)

        fmt.Print("Rating  : ")
        fmt.Scan(&(*pustaka)[i].Rating)
```

```

    }
}

/*
I.S. array pustaka berisi n data buku dan belum terurut
F.S. menampilkan buku dengan rating tertinggi
*/
func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        fmt.Println("Tidak ada data buku")
        return
    }

    maxIdx := 0

    for i := 1; i < n; i++ {
        if pustaka[i].Rating > pustaka[maxIdx].Rating {
            maxIdx = i
        }
    }

    fmt.Println("\n=== BUKU TERFAVORIT ===")
    fmt.Println("Judul   :", pustaka[maxIdx].Judul)
    fmt.Println("Penulis  :", pustaka[maxIdx].Penulis)
    fmt.Println("Penerbit :", pustaka[maxIdx].Penerbit)
    fmt.Println("Tahun   :", pustaka[maxIdx].Tahun)
    fmt.Println("Rating  :", pustaka[maxIdx].Rating)
}

/*
I.S. array pustaka berisi n data buku
F.S. array terurut menurun berdasarkan rating
Metode: Insertion Sort
*/
func UrutBuku(pustaka *DaftarBuku, n int) {

    for i := 1; i < n; i++ {

        temp := (*pustaka)[i]

        j := i - 1

        for j >= 0 && (*pustaka)[j].Rating < temp.Rating {
            (*pustaka)[j+1] = (*pustaka)[j]
            j--
        }

        (*pustaka)[j+1] = temp
    }
}

```

```

/*
I.S. pustaka sudah terurut berdasarkan rating
F.S. menampilkan maksimal 5 buku dengan rating tertinggi
*/
func Cetak5Teratas(pustaka DaftarBuku, n int) {

    fmt.Println("\n=== 5 JUDUL DENGAN RATING TERTINGGI ===")

    batas := 5

    if n < 5 {
        batas = n
    }

    for i := 0; i < batas; i++ {
        fmt.Printf("%d. %s (Rating %d)\n",
            i+1,
            pustaka[i].Judul,
            pustaka[i].Rating)
    }
}

```

```

/*
I.S. pustaka sudah terurut menurun berdasarkan rating
F.S. menampilkan salah satu buku dengan rating r
Metode: Binary Search
*/
func CariBuku(pustaka DaftarBuku, n int, r int) {

    left := 0
    right := n - 1

    for left <= right {

        mid := (left + right) / 2

        if pustaka[mid].Rating == r {

            fmt.Println("\n=== BUKU DITEMUKAN ===")
            fmt.Println("ID      :", pustaka[mid].ID)
            fmt.Println("Judul   :", pustaka[mid].Judul)
            fmt.Println("Penulis  :", pustaka[mid].Penulis)
            fmt.Println("Penerbit :", pustaka[mid].Penerbit)
            fmt.Println("Tahun    :", pustaka[mid].Tahun)
            fmt.Println("Eksemplar :", pustaka[mid].Eksemplar)
            fmt.Println("Rating   :", pustaka[mid].Rating)

            return
        }
    }
}

```

```

        // karena urut MENURUN
        if r > pustaka[mid].Rating {
            right = mid - 1
        } else {
            left = mid + 1
        }
    }

    fmt.Println("\nTidak ada buku dengan rating seperti itu")
}

func main() {
    var pustaka DaftarBuku
    var n int
    var ratingCari int
    fmt.Print("Jumlah buku: ")
    fmt.Scan(&n)
    DaftarkanBuku(&pustaka, n)
    CetakTerfavorit(pustaka, n)
    UrutBuku(&pustaka, n)
    Cetak5Teratas(pustaka, n)
    fmt.Print("\nRating yang dicari: ")
    fmt.Scan(&ratingCari)
    CariBuku(pustaka, n, ratingCari)
}

```

Output:

```

1 // Dharma Chandra Viriya
2 // 109082500052
3 // SIIF-13-02
4 package main
5
6 import "fmt"
7
8 const nMax = 7919
9
10 type Buku struct {
11     ID        string
12     Judul     string
13     Penulis  string
14     Penerbit  string
15     Eksemplar int
16     Tahun    int
17     Rating   float64
18 }
19
20 func main() {
21     f, err := os.Open("latihan2.csv")
22     if err != nil {
23         fmt.Println("Error opening file:", err)
24         return
25     }
26     defer f.Close()
27
28     // Read the CSV file
29     reader := csv.NewReader(f)
30     records, err := reader.ReadAll()
31     if err != nil {
32         fmt.Println("Error reading CSV file:", err)
33         return
34     }
35
36     // Print the data
37     for i, record := range records {
38         if i == 0 {
39             continue // Skip header row
40         }
41         buku := Buku{
42             ID:        record[0],
43             Judul:     record[1],
44             Penulis:  record[2],
45             Penerbit:  record[3],
46             Eksemplar: record[4],
47             Tahun:    record[5],
48             Rating:   record[6],
49         }
50         fmt.Println(buku)
51     }
52 }

```

Terminal Output:

```

Data Buku 1
ID : 1
Judul : MTK
Penulis : GuruMTK
Penerbit : Sekolah
Eksemplar : 12
Tahun : 2012
Rating : 88

Data Buku 2
ID : 2
Judul : English
Penulis : GuruEnglish
Penerbit : Sekolah
Eksemplar : 12
Tahun : 2013
Rating : 98

Data Buku 3
ID : 3
Judul : EtikaAI
Penulis : GuruEtikaAI
Penerbit : Sekolah
Eksemplar : 14
Tahun : 2022
Rating : 97

Data Buku 4
ID : 4
Judul : RPL
Penulis : GuruRPL

```



```
go -- main.go

Modul14 > InsertionSort > Latihan2 > main.go > CariBuku
1 // Dharma Chandra Viriya
2 // 109082500052
3 // S1IF-13-02
4 package main
5
6 import "fmt"
7
8 const nMax = 7919
9
10 type Buku struct {
11     ID      string
12     Judul   string
13     Penulis string
14     Penerbit string
15     Eksemplar int
16     Tahun   int
17     Rating  float64
18 }
19
20 func main() {
21     // Data Buku 4
22     b4 := Buku{ID: "4", Judul: "RPL", Penulis: "GuruRPL", Penerbit: "Sekolah", Eksemplar: 11, Tahun: 2017, Rating: 100}
23     // Data Buku 5
24     b5 := Buku{ID: "TKJ", Judul: "GuruTKJ", Penulis: "Sekolah", Penerbit: "13", Eksemplar: 2019, Tahun: 2000, Rating: 95}
25     // Data Buku 6
26     b6 := Buku{ID: "6", Judul: "Kalkulus", Penulis: "GuruKalkulus", Penerbit: "Sekolah", Eksemplar: 16, Tahun: 2002, Rating: 77}
27
28     // === BUKU TERFAVORIT ===
29     fmt.Println("=== BUKU TERFAVORIT ===")
30     for i := 0; i < len(buku); i++ {
31         fmt.Println(buku[i])
32     }
33 }
34
35 Data Buku 4
36 ID      : 4
37 Judul   : RPL
38 Penulis : GuruRPL
39 Penerbit : Sekolah
40 Eksemplar : 11
41 Tahun   : 2017
42 Rating  : 100
43
44 Data Buku 5
45 ID      : TKJ
46 Judul   : GuruTKJ
47 Penulis : Sekolah
48 Penerbit : 13
49 Eksemplar : 2019
50 Tahun   : 2000
51 Rating  : 95
52
53 Data Buku 6
54 ID      : 6
55 Judul   : Kalkulus
56 Penulis : GuruKalkulus
57 Penerbit : Sekolah
58 Eksemplar : 16
59 Tahun   : 2002
60 Rating  : 77
61
62 === BUKU TERFAVORIT ===
63 Judul   : RPL
64 Penulis : GuruRPL
65 Penerbit : Sekolah
66 Tahun   : 2017
67 Rating  : 100
```

```
go -- main.go

Modul14 > InsertionSort > Latihan2 > main.go > CariBuku
1 // Dharma Chandra Viriya
2 // 109082500052
3 // S1IF-13-02
4 package main
5
6 import "fmt"
7
8 const nMax = 7919
9
10 type Buku struct {
11     ID      string
12     Judul   string
13     Penulis string
14     Penerbit string
15     Eksemplar int
16     Tahun   int
17     Rating  float64
18 }
19
20 func main() {
21     // Data Buku 4
22     b4 := Buku{ID: "4", Judul: "RPL", Penulis: "GuruRPL", Penerbit: "Sekolah", Eksemplar: 11, Tahun: 2017, Rating: 100}
23     // Data Buku 5
24     b5 := Buku{ID: "TKJ", Judul: "GuruTKJ", Penulis: "Sekolah", Penerbit: "13", Eksemplar: 2019, Tahun: 2000, Rating: 95}
25     // Data Buku 6
26     b6 := Buku{ID: "6", Judul: "Kalkulus", Penulis: "GuruKalkulus", Penerbit: "Sekolah", Eksemplar: 16, Tahun: 2002, Rating: 77}
27
28     // === BUKU TERFAVORIT ===
29     fmt.Println("=== BUKU TERFAVORIT ===")
30     for i := 0; i < len(buku); i++ {
31         fmt.Println(buku[i])
32     }
33
34     // === $ JUDUL DENGAN RATING TERTINGGI ===
35     fmt.Println("=== $ JUDUL DENGAN RATING TERTINGGI ===")
36     for i := 0; i < len(buku); i++ {
37         fmt.Println(buku[i].Judul, "Rating:", buku[i].Rating)
38     }
39
40     // Rating yang dicari: 100
41     rating := 100
42     for i := 0; i < len(buku); i++ {
43         if buku[i].Rating == rating {
44             // === BUKU DITEMUKAN ===
45             fmt.Println("=== BUKU DITEMUKAN ===")
46             fmt.Println(buku[i])
47         }
48     }
49 }
50
51 Data Buku 4
52 ID      : 4
53 Judul   : RPL
54 Penulis : GuruRPL
55 Penerbit : Sekolah
56 Tahun   : 2017
57 Rating  : 100
58
59 Data Buku 5
60 ID      : TKJ
61 Judul   : GuruTKJ
62 Penulis : Sekolah
63 Penerbit : 13
64 Tahun   : 2019
65 Rating  : 95
66
67 Data Buku 6
68 ID      : 6
69 Judul   : Kalkulus
70 Penulis : GuruKalkulus
71 Penerbit : Sekolah
72 Tahun   : 2002
73 Rating  : 77
74
75 === $ JUDUL DENGAN RATING TERTINGGI ===
76 1. RPL (Rating 100)
77 2. English (Rating 98)
78 3. EtikaAI (Rating 97)
79 4. GuruTKJ (Rating 95)
80 5. MTK (Rating 88)
81
82 Rating yang dicari: 100
83
84 === BUKU DITEMUKAN ===
85 ID      : 4
86 Judul   : RPL
87 Penulis : GuruRPL
88 Penerbit : Sekolah
89 Tahun   : 2017
90 Rating  : 100
```

Deskripsi Program:

Program ini digunakan untuk mengelola dan mengolah data buku pada sebuah perpustakaan. Setiap data buku terdiri atas ID buku, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating. Program menerima sejumlah data buku sebagai masukan, kemudian menampilkan informasi buku terfavorit, lima buku dengan rating tertinggi, serta melakukan pencarian buku berdasarkan rating yang dimasukkan oleh pengguna. Secara teknis, data buku direpresentasikan menggunakan struktur data 'Buku', sedangkan seluruh koleksi buku disimpan dalam array 'DaftarBuku'. Proses pengisian data dilakukan melalui prosedur 'DaftarkanBuku()' yang membaca seluruh atribut buku dan menyimpannya ke dalam array. Untuk menentukan buku terfavorit, program melakukan penelusuran terhadap seluruh data dan mencari buku yang memiliki nilai rating paling tinggi. Informasi yang ditampilkan meliputi judul, penulis, penerbit, tahun terbit, dan rating buku tersebut. Pengurutan data dilakukan menggunakan algoritma Insertion Sort yang diimplementasikan pada fungsi 'UrutBuku()'. Algoritma ini bekerja dengan menganggap sebagian data telah terurut, kemudian mengambil satu elemen dan menyisipkannya ke posisi yang sesuai pada bagian yang sudah terurut tersebut. Pada program ini, proses penyisipan dilakukan berdasarkan nilai rating sehingga setelah pengurutan selesai, data buku tersusun secara menurun (descending) dari rating tertinggi ke rating terendah. Penggunaan Insertion Sort sesuai dengan spesifikasi soal dan efektif digunakan untuk memahami konsep pengurutan berbasis penyisipan elemen. Setelah data terurut, program menampilkan lima buku dengan rating tertinggi melalui fungsi 'Cetak5Teratas()'. Karena data telah diurutkan secara menurun, lima buku pertama pada array secara otomatis merupakan buku-buku dengan rating tertinggi. Jika jumlah buku kurang dari lima, program hanya menampilkan seluruh buku yang tersedia tanpa menimbulkan kesalahan. Untuk proses pencarian, program menggunakan algoritma Binary Search yang diimplementasikan pada fungsi 'CariBuku()'. Metode ini memanfaatkan kondisi data yang sudah terurut sehingga pencarian dapat dilakukan dengan membagi ruang pencarian menjadi dua bagian secara berulang. Pada setiap iterasi, rating yang dicari dibandingkan dengan elemen tengah array. Jika nilai yang dicari lebih besar atau lebih kecil dari elemen tengah, pencarian dilanjutkan hanya pada bagian array yang masih mungkin mengandung data tersebut. Apabila ditemukan buku dengan rating yang sesuai, seluruh informasi buku akan ditampilkan. Jika tidak ditemukan, program menampilkan pesan bahwa tidak ada buku dengan rating tersebut. Dari sisi kompleksitas, proses pencarian buku terfavorit memerlukan waktu $O(n)$ karena harus memeriksa seluruh data. Proses pengurutan menggunakan Insertion Sort memiliki kompleksitas waktu $O(n^2)$, sedangkan proses pencarian menggunakan Binary Search memiliki kompleksitas $O(\log n)$. Kombinasi kedua algoritma tersebut memungkinkan program tidak hanya mengurutkan data sesuai ketentuan soal, tetapi juga melakukan pencarian dengan lebih efisien setelah data berhasil diurutkan.

DAFTAR PUSTAKA